



PROJECT SYNOPSIS

“AI-BASED STUDENT SKILL GAP ANALYZER”

PROJECT TEAM

1.K R Nandeeshha – 1IC24AI012

PROJECT GUIDE

Prof Sumaya Fathima

Table of Contents

1. ABSTRACT
2. PROBLEM STATEMENT
3. SYSTEM REQUIREMENTS
4. METHODOLOGY
5. EXPECTED OUTCOMES

1. ABSTRACT

1.1 Background and Context

- The transition from undergraduate academic institutions to the technology industry represents a significant challenge for modern engineering graduates.
- In today's corporate landscape, technology recruiters prioritize hands-on development competencies, system architecture understanding, and practical problem-solving capabilities over pure theoretical marks.
- A persistent gap exists between traditional academic curricula and rapidly evolving industrial skill requirements, leading to high rejection rates during technical recruitment drives.

1.2 Proposed System Summary

- To resolve this critical divide, this project proposes the '**AI-Based Student Skill Gap Analyzer**', an automated platform designed to evaluate a student's technical capabilities against real-time industry demands.
- The platform leverages Artificial Intelligence (AI), Natural Language Processing (NLP), and Machine Learning (ML) to ingest heterogeneous student datasets without relying on subjective self-assessment surveys.
- The system analyzes unstructured resume documents, academic transcripts, and open-source codebase contributions (e.g., GitHub repositories) to construct a verified competency profile.

1.3 Industry Vectoring and AI Recommendations

- Simultaneously, the platform continuously processes contemporary job descriptions across engineering domains to build a dynamic vector database of industry expectations.
- By utilizing Retrieval-Augmented Generation (RAG) pipelines and semantic vector matching, the system calculates precise skill gaps in areas such as algorithmic optimization, database management, and cloud infrastructure.
- The system delivers hyper-personalized, step-by-step remediation roadmaps, recommending specific micro-credentials, coding handbooks, and open-source contributions to maximize graduate employability.

1.4 Key Objectives

- Automate the extraction and verification of technical skills from student portfolios and source code.
- Maintain an updated industry baseline using automated job market scraping and vector indexing.
- Generate grounded, hallucination-free upskilling pathways using RAG architecture.

2. PROBLEM STATEMENT

2.1 Systemic Limitations in Modern Academic Environments

- **Syllabus Updating Cycles:** Academic curriculum approval lifecycles span multiple years, whereas software tools, frameworks, and deployment standards transform on an annual basis.
- **Lack of Production Exposure:** University coursework emphasizes theoretical computing logic but provides minimal exposure to production-grade ecosystems, such as scalable cloud databases, API microservices, and containerized deployments.
- **Generalized Evaluation Metrics:** Standard institutional evaluations rely on uniform written examinations and fixed laboratory programs, failing to account for individual learning curves or niche career paths.
- **Absence of Real-Time Feedback:** Students remain unaware of specific technical deficiencies until they face repeated rejections during technical recruitment interviews.
- **Scalability Limits of Manual Counseling:** Academic career guidance counselors cannot manually map granular, rapidly shifting market requirements to hundreds of individual student portfolios.

2.2 Comparative Analysis of Assessment Paradigms

EVALUATION METRIC	TRADITIONAL ACADEMIC METHODS	PROPOSED AI SKILL GAP ANALYZER
Primary Data Source	Written exams & basic lab marks	Resumes, GitHub code, dynamic job postings
Verification Style	Theoretical tests & self-reports	AST code analysis & NLP vector matching
Market Alignment	Static syllabus updated multi-yearly	Live vector database (Pinecone/Milvus)
Feedback Type	Percentage marks & letter grades	Actionable step-by-step upskilling roadmap
Scalability	Constrained by faculty availability	Fully automated, continuous cloud service

2.3 Problem Statement Summary

Therefore, there exists a critical need for an automated, AI-driven evaluation system that objectively parses student technical portfolios, quantifies skill deficiencies against live industrial standards, and generates verified upskilling pathways to enhance graduate readiness.

3. SYSTEM REQUIREMENTS

3.1 Hardware Specifications

- **Development Environment:**
 - Processor: Minimum Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent.
 - System Memory: Minimum 16 GB RAM (necessary for local vector database processing and concurrent microservice execution).
 - Storage: Minimum 256 GB Solid State Drive (SSD) for rapid read/write operations during dataset processing.
- **Deployment Infrastructure:**
 - Production Cloud Server: Multi-core virtualized CPU instances with scalable RAM configuration.
 - Network: High-bandwidth connection supporting low-latency REST API requests.

3.2 Software Environment & Technology Stack

SYSTEM LAYER	TECHNOLOGY SELECTED	FUNCTIONAL PURPOSE
Frontend Interface	Next.js (Web) & React Native Expo (Mobile)	Cross-platform user dashboard, resume upload, path visualization
Backend Services	Python, FastAPI	Asynchronous API layer & multi-agent orchestration
Primary Database	MongoDB Atlas & Supabase	Hybrid database storing unstructured resumes and structured auth data
Vector Engine	Pinecone / Milvus	High-dimensional embedding store for semantic search
Cloud Hosting	Vercel (Frontend) & Render (Backend)	Edge network hosting and scalable backend service deployment

3.3 Functional Requirements

- **User Profile Management:** Secure authentication and portfolio submission interface.
- **Automated Resume Parsing:** Extraction of key technical terms, projects, and credentials.
- **Job Market Vector Indexing:** Automated scraping and vectorization of real-time job descriptions.
- **Comparative Analytics Engine:** Sub-second calculation of skill gaps using vector metrics.

4. METHODOLOGY (PART I: INGESTION & ANALYSIS)

4.1 Multi-Agent Architecture Overview

The proposed platform adopts a decoupled multi-agent architectural model. Operations are divided across dedicated, context-aware agents managed by a FastAPI backend:

- **Profiler Agent:** Responsible for parsing student submissions, extracting skill terms, and validating code repositories.
- **Industry Agent:** Continuously monitors, categorizes, and indexes current job market postings into high-dimensional vector spaces.
- **Mentor Agent:** Executes semantic comparisons and constructs structured learning pathways using Retrieval-Augmented Generation (RAG).

4.2 Step-by-Step Implementation Workflow

Step 1: Data Ingestion & Input Preprocessing

- The student submits their academic details, resume document (PDF/Word format), and public GitHub profile handle.
- The system performs text extraction, tokenization, lemmatization, and noise reduction on the submitted resume.

Step 2: Practical Skill Verification via Source Code AST Parsing

- To eliminate reliance on unverified self-reported skills, the system interfaces with public GitHub APIs.
- The Profiler Agent parses the Abstract Syntax Trees (AST) and call graphs of the student's code repositories.
- This process verifies practical programming proficiency in data structures, API routing, and database integrations.

Step 3: Industry Demand Vector Mapping

- Automated data ingestion scripts gather job listings from major tech hiring portals.
- Named Entity Recognition (NER) models extract required technologies, frameworks, and experience levels.
- Dense vector embeddings are generated using Large Language Models and indexed into Pinecone/Milvus databases.

4. METHODOLOGY (PART II: RECOMMENDATION & EXECUTION)

4.3 Step-by-Step Implementation Workflow (Continued)

Step 4: Vector Distance & Skill Gap Quantification

- The platform computes mathematical similarity between the student's skill vector (S) and the target industry job requirement vector (J).
- Cosine similarity calculations determine the degree of alignment:

Cosine Similarity: $\cos(\theta) = (S \cdot J) / (||S|| \times ||J||)$

- Discrepancies between vectors are categorized into Critical Skill Gaps and Secondary Enhancements.

Step 5: RAG-Guided Pathway Generation via Directed Acyclic Graphs (DAG)

- To prevent LLM hallucination, the Mentor Agent queries the vector database for verified, up-to-date educational materials.
- The system structures learning paths as Directed Acyclic Graphs (DAGs), where nodes represent modules and edges represent prerequisites.
- Topological sorting algorithms arrange topics to ensure foundational concepts (e.g., SQL syntax) are mastered prior to advanced topics (e.g., database indexing).

4.4 Performance Evaluation Parameters

PARAMETER	EVALUATION METHODOLOGY	TARGET BENCHMARK
Extraction Precision	Named Entity Recognition accuracy vs. manual resume annotations	> 92% Precision
Matching Accuracy	Cosine similarity vs. expert-curated skill sets	> 90% Alignment
API Latency	End-to-end FastAPI execution response time	< 800 ms
Path Groundedness	Context verification of generated learning recommendations	100% Verified Links

5. EXPECTED OUTCOMES & DELIVERABLES

5.1 Expected System Outcomes

- **Automated Skill Dashboard:** An objective digital interface providing granular visibility into a student's technical proficiency matched against live industry roles.
- **Hyper-Personalized Upskilling Roadmaps:** Step-by-step learning paths that route students to verified micro-credentials (e.g., IBM SkillsBuild), competitive coding platforms (e.g., Kaggle), and handbooks (e.g., freeCodeCamp).
- **Gamified Progress Tracking:** Milestone tracking and completion analytics to sustain student engagement during self-paced learning.
- **Enhanced Graduate Employability:** Direct reduction in technical interview rejection rates through targeted skills alignment.

5.2 Primary Project Deliverables

S. NO.	DELIVERABLE MODULE	TECHNICAL DESCRIPTION
1	Web & Mobile User Portal	Next.js dashboard and React Native Expo app for profile submission and path tracking.
2	FastAPI Orchestration Backend	Asynchronous Python backend running the Profiler, Industry, and Mentor AI agents.
3	RAG Vector Database Pipeline	Pinecone/Milvus index storing updated job descriptions and verified learning resources.
4	AST Code Verification Engine	GitHub repository parser evaluating practical coding experience from source commits.

CONCLUSION

The AI-Based Student Skill Gap Analyzer bridges the gap between academic learning and industry expectation: through objective, data-driven skill profiling. By combining a Multi-Agent architecture with a RAG pipeline, the system delivers real-time, accurate market alignment without outdated advice. The platform provides personalized, step-by-step learning roadmaps using micro-credentials, coding challenges, and repository evaluation. Overall, it functions as a